BiOCamLib

BiOCamLib is the OCaml foundation upon which a number of the bioinformatics tools I developed are built.

It mostly consists of a library — you'll need to clone this repository if you want to manually compile other programs I've developed, notably SiNPle or KPop. You might also use the library for your own programs, if you are familiar with OCaml and patient enough to read the code.

As a bonus, BiOCamLib comes bundled with a few programs:

- `RC`, which can efficiently compute the reverse complement of (possibly very long) sequences. Each sequence should be input on a separate line — lines are processed one by one and not buffered.
- `Octopus`, which is a high-throughput program to compute the transitive closure of strings. This is useful to cluster things by agglomeration schemes.
- `Parallel`, which allows you to split and process an input file chunk-wise using the reader/workers/writer model implemented in `BiOCamLib.Tools.Parallel`. You can see it as a demonstration of the capabilities of the library, but I also often use it as a useful tool to solve real-life problems, be they bioinformatics or not. A number of high-throughput real-life examples can be found in the KPop README.
- `FASTools`, which is a Swiss-knife tool for the manipulation of FASTA/FASTQ files. It supports all formats (FASTA, single- and paired-end FASTQ, interleaved FASTQ) and a simpler tabular format whereby FASTA/FASTQ records are represented as tab-separated lines. It facilitates format interconversions and other manipulations.
- `TREx`, which finds exact tandem repeats in all the sequences of a FASTA file. The output is a tab-separated table of repeats with their position, length, and unit, suitable for downstream filtering or annotation.
- `Cophenetic`, which reads a Newick tree on standard input and writes the cophenetic-distance matrix between every pair of labelled nodes (leaves and internal alike) to standard output. By default the matrix carries shortest-path distances along branch lengths; an option switches to longest-path distances, which is the right convention when branch lengths encode partial quantities (e.g. allele frequencies).
- `NJ`, which goes the other way from `Cophenetic`: it reads a matrix of pairwise distances and writes the neighbour-joining tree those distances imply, in Newick format. The tree is unrooted, as neighbour joining's is; an option re-roots it at the midpoint of its longest tip-to-tip path, which is the usual stand-in for an outgroup when none is available. A matrix that is not quite symmetric is averaged across the diagonal by default, and the negative branch lengths a non-additive matrix yields can be kept, flattened to zero, or made an error.
- `Yggdrasill`, which builds a phylogenetic tree from a register of weighted splits. It loads a `.PhyloSplits` file (binary or tabular), greedy-filters the splits for pairwise compatibility in order of decreasing weight, and emits a Newick tree assembled via the Buneman construction. The incompatible residual is retained for further inspection or iterative refinement, and a dropped-weight ratio is emitted to stderr as a measure of how tree-like the input split system is. This is the natural downstream tool for KPop's `KPopTwistDB --splits` output.
- `AnnoTools`, which is a Swiss-knife tool for the manipulation of annotation files. It can read and write GFF3, GTF, and GenBank files, attach a multi-FASTA reference, validate that an annotation is consistent with the reference, select features and extract their DNA or protein sequences. Native lossless formats (either a compact binary archive that loads orders of magnitude faster than reparsing the source, or a lossless set of tab-separated tables you can `diff` or manipulate with regular Unix tools) are also supported.

Installing `RC`, `Octopus`, `Parallel`, `FASTools`, `TREx`, `Cophenetic`, `NJ`, `Yggdrasill`, and `AnnoTools`

⚠️ Note that the only operating systems we officially support are Linux and MacOS. ⚠️

There are several possible ways of installing the software on your machine: through `conda`; by downloading pre-compiled binaries (Linux x86_64 and MacOS only); or manually.

Conda channel

BiOCamLib can be installed from the `bioconda` channel as package `biocamlib`. Just type

```
conda install -c bioconda biocamlib
```

or the equivalent command obtained replacing `conda` with `mamba` or `micromamba`, depending on which program you habitually use.

Pre-compiled binaries

You can download pre-compiled binaries for Linux and MacOS x86_64 from our releases. After doing so, just copy or move them to a directory which is accessible from your PATH.

For instance, supposing that you've downloaded programs to directory `~/.local/bin/`, in order to make them accessible from everywhere you'll have to execute a command such as

```
export PATH=~/.local/bin:$PATH
```

or add it to one of your login scripts (such as `~/.bashrc` or similar for `bash`).

Note that the binaries are generated according to the recipe described here.

Manual install

Alternatively, you can install `RC`, `Octopus`, `Parallel`, `FASTools`, `TREx`, `Cophenetic`, `NJ`, `Yggdrasill`, and `AnnoTools` manually by cloning and compiling the BiOCamLib sources. You'll need an up-to-date distribution of the OCaml compiler and the Dune package manager for that. Both can be installed through OPAM, the official OCaml distribution system. Once you have a working OPAM distribution you'll also have a working OCaml compiler, and Dune can be installed with the command

```
opam install dune
```

if it is not already present. Make sure that you install OCaml version 4.12 or later.

Then go to the directory into which you have downloaded the latest BiOCamLib sources, and type

```
./BUILD
```

This should generate the executables `RC`, `Octopus`, `Parallel`, `FASTools`, `TREx`, `Cophenetic`, `NJ`, `Yggdrasill`, and `AnnoTools`. Copy them to some favourite location in your PATH, for instance `~/.local/bin`.

Using `RC`

`RC` inputs sequences from standard input and outputs their reverse complement to standard output, one sequence at the time. Hence, `RC` can be conveniently used in a subprocess. For example, the command

```
echo GAtTaCA | RC
```

would produce `TGtAaTC`. The whole IUPAC nucleotide alphabet is complemented, not just `[ACGTacgt]` — an ambiguity code is mapped to the code for the complementary set, so `R` (`A`/`G`) becomes `Y` (`C`/`T`), `B` becomes `V`, and the self-complementary `S`, `W` and `N` are left alone. `U` complements to `A`. Case is always preserved. Characters outside the nucleotide alphabet, gaps included, are output unmodified, so sequence validation and linting must still be performed elsewhere whenever they are necessary.

Command line options

This is the full list of command line options available for the program `RC`. You can visualise the list by typing

```
RC -h
```

in your terminal. You will see a header containing information about the version:

```
This is RC version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2023-2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
RC [OPTIONS]
```

Algorithm

Option	Argument(s)	Effect	Note(s)
`-C` `--no-complement`		do not base-complement the sequence	default= *base-complement*

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `Octopus`

`Octopus` reads from its standard input equivalence relations, one set of relations per line. Each line consists of a set of strings separated by whitespace; if any two labels appear on the same line, they are considered to belong to the same equivalence class. When all the input has been parsed, `Octopus` outputs all the labels seen in the input sorted according to their equivalence class — each line contains one equivalence class, with its member string labels separated by a `\t` character. The order in which classes appear is kept, but elements within the class will be lexicographically sorted. For example, the command

```
(cat <<___
A duh
  b  C
c f e
 duh zz x
b c

___
) | Octopus
```

will result in the output

```
A       duh     x       zz
C       b       c       e       f
```

(tab-separated).

Command line options for `Octopus`

This is the full list of command line options available for the program `Octopus`. You can visualise the list by typing

```
Octopus -h
```

in your terminal. You will see a header containing information about the version:

```
This is Octopus version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2016-2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
Octopus [OPTIONS]
```

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `Parallel`

`Parallel` implements a subset of the functionality of GNU `parallel`, but is a compiled binary (hence potentially faster) and does not require you to install PERL or any other dependencies. It also has a much simpler interface. You can use it to transparently split a multi-line file into blocks having a specified size, which are then fed as standard input to a pool of worker threads. The number of threads is specified by the user, and the next block to be processed is assigned to the first thread that becomes available. The results of the processing of the blocks are concatenated in the order of the original blocks. If the number of threads is not specified, as many are used as the number of available processing units/CPU cores (as determined by the `nproc` command).

A typical bioinformatic use case might be something like

```
cat LargeFile.fasta | Parallel -l 1000 -- awk '{if ($0~"^>") print; else print toupper($0)}'
```

which would spawn as many workers as the number of available cores, split the input FASTA file into blocks of 1,000 lines each and process each block through `awk` to capitalise the sequence.

Keep in mind that there is little point in parallelising a job if sending/receiving data to/from it takes more time than the computation itself!

Command line options for `Parallel`

This is the full list of command line options available for the program `Parallel`. You can visualise the list by typing

```
Parallel -h
```

in your terminal. You will see a header containing information about the version:

```
This is Parallel version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2019-2026 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
Parallel [OPTIONS] -- [COMMAND TO PARALLELIZE AND ITS OPTIONS]
```

Command to parallelize

Option	Argument(s)	Effect	Note(s)
`--`		consider all the subsequent parameters as the command to be executed in parallel. At least one command must be specified	*(mandatory)*

Input/Output

Option	Argument(s)	Effect	Note(s)
`-l` `--lines-per-block`	*<positive_integer>*	number of lines to be processed per block	default= *10000*
`-i` `--input`	*<input_file>*	name of input file	default= */dev/stdin*
`-o` `--output`	*<output_file>*	name of output file	default= */dev/stdout*

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-t` `--threads`	*<positive_integer>*	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= *4*
`-v` `--verbose`		set verbose execution	default= *quiet execution*
`-d` `--debug`		output debugging information	default= *false*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `FASTools`

`FASTools` allows you to manipulate FASTA/FASTQ files by offering a rich set of tools to convert FASTA/FASTQ records to/from a tab-separated format. Together with other programs such as `awk`, that allows a large set of operations to be implemented effortlessly.

For example:

```
cat Sequences.fasta | FASTools | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

puts each FASTA record in file `Sequences.fasta` on a single line, with the sequence name and the sequence being separated by a `'\t'` character. The `awk` part then reads each line, replaces the sequence with its first half, and re-outputs the sequence as a FASTA record.

Due to default command line option values, the command is actually equivalent to

```
cat Sequences.fasta | FASTools c -F | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

and, if one uses forms accepting a direct rather than a piped input, to

```
FASTools -f Sequences.fasta | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

or to

```
FASTools c -f Sequences.fasta | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

Similar command line options are provided to manipulate as single lines FASTQ and tabular formats rather than FASTA. In detail:

- Option `-f <FASTA_file>` operates on a FASTA file given as an explicit argument, while `-F` processes the same kind of file piped into the program as standard input
- Option `-s <FASTQ_file>` operates on a FASTQ file containing single-end reads given as an explicit argument, while `-S` processes the same kind of file piped into the program as standard input
- Option `-p <FASTQ_file_1> <FASTQ_file_2>` operates on two FASTQ file separately containing first and second ends of paired-end reads given as an explicit argument, while `-P` processes an interleaved FASTQ file containing alternating first and second ends of paired-end reads piped into the program as standard input
- Option `-t <tabular_file>` operates on a tabular file containing FASTA/FASTQ records compacted on a single line by FASTools given as an explicit argument, while `-T` processes the same kind of file piped into the program as standard input. Note that there are exactly three possible tabular formats recognised by FASTools:
    1. Lines of the form `<read_name> '\t' <read_sequence>`, corresponding to a compacted FASTA record
    2. Lines of the form `<read_name> '\t' <read_sequence> '\t' <read_qualities>`, corresponding to a compacted single-end FASTQ record
    3. Lines of the form `<read_name_1> '\t' <read_sequence_1> '\t' <read_qualities_1> '\t' <read_name_2> '\t' <read_sequence_2> '\t' <read_qualities_2>`, corresponding to a compacted paired-end FASTQ record.

Several command switches exist that natively perform operations on sequence/qualities within `FASTools`, with no need to write any additional external code:

- Option `c` or `-c`, a shorthand for `compact`, will group each FASTA/FASTQ record on a single tab-separated line. It is the default
- Option `e` or `-e`, a shorthand for `expand`, will split tabular records into multi-line FASTA/FASTQ records. It is the inverse of `c`. Note that it is perfectly fine to apply option `e` to an existing input which is already in FASTA/FASTQ format, in which case `FASTools` will normalise the input by putting each sequence on a single line, which is how `FASTools` outputs sequences. Linting filters (option `-l`) can also be used to further normalise the sequence

- Option `m <regexp>` or `-m <regexp>`, a shorthand for `match <regexp>`, will select FASTA/FASTQ records whose name match ''. Note that `<regexp>` must be defined according to https://ocaml.org/api/Str.html
- Option `r` or `-r`, a shorthand for `revcom`, will reverse-complement the sequence, and reverse the qualities if present, of FASTA/FASTQ records
- Option `d` or `-d`, a shorthand for `dropq`, will drop qualities from FASTA/FASTQ records, turning a FASTQ into a FASTA file and having no effect on a FASTA file.

Repeated command line options can be seen as different commands that are executed in order of specification. For instance,

```
FASTools m "^CONTIG_1$" -f Assembly_1.fasta m "^CONTIG_42$" -f Assembly_2.fasta
```

will select sequence `CONTIG_1` from file `Assembly_1.fasta` and sequence `CONTIG_42` from file `Assembly_2.fasta`, outputting one after the other. Note however that each `c` / `e` / `m` / `r` / `d` option will input and output precisely one file, so option `c` in

```
FASTools e -f Multiline.fasta c
```

will not have any effect — the result will be a FASTA file with sequences on the same line, not a tabular one. To perform on the same input more than one transformation requiring I/O, you'll have to pipe multiple `FASTools` commands one after the other, as in

```
FASTools e -f Multiline.fasta | FASTools c
```

which will give the expected result.

A few more examples:

- ```
  FASTools e -p <(zcat Sample76_1.fq.gz) <(zcat Sample76_2.fq.gz)
  ```

  will interleave compressed files, while

- ```
  FASTools -s Reads.fastq | shuf | awk '((NR-1)%10==0)' | FASTools e -T
  ```

  will randomly select one read in 10. Note that the corresponding version with paired-end files in input, namely:

- ```
  FASTools -p Reads_1.fastq Reads_2.fastq | shuf | awk '((NR-1)%10==0)' | FASTools e -T
  ```

  will still work correctly and produce an interleaved output.

Finally, you can combine `FASTools` with `Parallel` to distribute computation across different CPUs. For instance, the command:

```
FASTools -f Sequences.fasta | Parallel -- awk -F '\t' '{print
        ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

will still cut sequences in half as the first example, but it will do so by splitting the input file compacted by FASTools into blocks of 10,000 records each and by distributing the computation among all the available CPU cores.

Command line options for `FASTools`

This is the full list of command line options available for the program `FASTools`. You can visualise the list by typing

```
FASTools -h
```
```

in your terminal. You will see a header containing information about the version:

```
This is FASTools version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2022-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
FASTools [OPTIONS]
```

Working mode. Executed delayed in order of specification, default='compact'.

Option	Argument(s)	Effect	Note(s)
`compact` `-c` `--compact`		put each FASTA/FASTQ record on one tab-separated line (default mode)	
`expand` `-e` `--expand`		split each tab-separated line into one or more FASTA/FASTQ records	
`revcom` `-r` `--revcom`		reverse-complement sequences (and reverse qualities if present) in FASTA/FASTQ records or tab-separated lines	
`dropq` `-d` `--dropq`		drop qualities in FASTA/FASTQ records or tab-separated lines	
`match` `-m` `--match`	*<regexp>*	select sequence names matching the specified regexp in FASTA/FASTQ records or tab-separated lines. The regexp must be defined according to <https://ocaml.org/api/Str.html>. For paired-end files, the pair matches when at least one name matches.	
`rename` `-R` `--rename`	*<regexp>* *<replacement>*	replace with the provided pattern all the instances of the specified regexp occurring in the sequence names of FASTA/FASTQ records or tab-separated lines. The regexp must be defined according to <https://ocaml.org/api/Str.html>. Replacement expressions can contain identifiers \1 ... \9 for the groups matched by the regular expression; \0 represents the full match	

Input/Output. Executed delayed in order of specification, default='-F'.

Option	Argument(s)	Effect	Note(s)
`-f` `--fasta`	*<fasta_file_name>*	process FASTA input file containing sequences	
`-F`		process FASTA sequences from standard input	
`-s` `--single-end`	*<fastq_file_name>*	process FASTQ input file containing single-end sequencing reads	
`-S`		process single-end FASTQ sequencing reads from standard input	
`-p` `--paired-end`	*<fastq_file_name1>* *<fastq_file_name2>*	process FASTQ input files containing paired-end sequencing reads	

Option	Argument(s)	Effect	Note(s)
`-P`		process interleaved FASTQ sequencing reads from standard input	
`-t` `--tabular`	*<tabular_file_name>*	process input file containing FAST[A|Q] records as tab-separated lines	
`-T`		process FAST[A|Q] records in tabular form from standard input	
`-l` `--linter`	*'none'|'DNA'|'dna'|'protein'*	sets linter for sequence. All non-base (for DNA) or non-AA (for protein) characters are converted to unknowns	default= *none*
`--linter-keep-lowercase`	*'true'|'false'*	sets whether the linter should keep lowercase DNA/protein characters appearing in sequences rather than capitalise them	default= *false*
`--linter-keep-dashes`	*'true'|'false'*	sets whether the linter should keep dashes appearing in sequences rather than convert them to unknowns	default= *false*
`-o` `--output`	*<output_file_name>*	set the name of the output file. Files are kept open, and it is possible to switch between them by repeatedly using this option. Use '/dev/stdout' for standard output	default= */dev/stdout*
`-O` `--paired-end-output`	*<output_file_name_1>* *<output_file_name_2>*	set the names of paired-end FASTQ output files. Files are kept open, and it is possible to switch between them by repeatedly using this option. Use '/dev/stdout' for standard output	default= */dev/stdout*
`--flush` `--flush-output`		flush output after each record (global option)	default= *do not flush*

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-v` `--verbose`		set verbose execution (global option)	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `AnnoTools`

`AnnoTools` allows you to parse a genomic annotation and convert it from/to different widely used formats. It manipulates a single in-memory genome-annotation register through a CLI-driven action stream. The action stream is built up across the command line (one switch corresponding to one action) and executed in order at the end — the same pattern used by `FASTools`. Supported formats are GFF3, GTF, and GenBank; multi-FASTA files can be read to add reference sequences to the annotation. Two of the formats carry their own sequence and so need no separate FASTA: a GenBank file's `ORIGIN` block, and a GFF3 file's `##FASTA` section — a standard directive saying that the rest of the file is sequence. `AnnoTools` reads and writes both, so a GenBank record survives a trip through GFF3 whole, sequence included. The resulting contents of the register can be written back to any of the three formats or serialised to a compact `.Annotation` binary archive.

A typical round-trip looks like

```
AnnoTools --from-gff3 chr22.gff3 --to-gtf chr22.gtf
```

which loads `chr22.gff3` into a fresh register and writes it back out as a GTF file. The `--from-...` switches default to *replace* semantics — they wipe the current register and start over. The `--annotation` switch lets you instead say *add*, which extends the existing register with the new file's features:

```
AnnoTools --from-gff3 part1.gff3 -a add gff3 part2.gff3 --to-gff3 merged.gff3
```

Once a register has been built, options `-o` / `-i` let you cache it for later runs:

```
AnnoTools --from-gff3 huge.gff3 --from-fasta huge.fa -o huge        # loads huge.gff3 and writes
        huge.Annotation in binary format
AnnoTools -i huge --to-gtf huge.gtf                                 # reloads binary and writes
        contents to huge.gtf
```

Reloading is dominated by I/O (no parsing), which on GENCODE-class inputs can be orders of magnitude faster than reparsing the source.

Parsing and validating annotation files

Annotation files are notoriously fragile, with each major format having its quirks and a number of sometimes quite divergent dialects. `AnnoTools` provides configurable parsing and validation by associating each format to a description of the same in terms of one of more *hierarchies* of features. Hierarchies can be redefined by the user, providing a general way to describe and validate content even when the annotation schema used is not entirely standard.

`AnnoTools` ships with default annotation schemas for Genbank and GTF, according to their respective standards. GFF3 input is interpreted under a built-in *standard* hierarchy by default. `AnnoTools` also provides a second built-in GFF3 dialect for the GENCODE schema, which collapses every transcript biotype into the single type `transcript` and adds the `stop_codon_redefined_as_selenocysteine` child of `CDS`; switch to it via

```
AnnoTools --dialect gff3 gencode --from-gff3 gencode.v47.basic.annotation.gff3 -o gencode_v47
```

The `--dialect` and `--hierarchy` overrides are *sticky*: they apply to every subsequent input operation in the named format until another `--dialect` or `--hierarchy` replaces them. To revert to a format's default, just say `--dialect <fmt> standard`. A custom hierarchy can be pinned via `--hierarchy <fmt> "<S-expression>"`.

Once loaded, the annotation (and an annotation combined with a reference sequence when the latter has been added to the register) can be validated. So

```
AnnoTools --from-gff3 chr22.gff3 --from-fasta chr22.fa --validate
```

loads the annotation, attaches the matching reference, and runs every consistency check implemented:

- `--validate-sequences-present` (every annotated sequence name appears in the reference)
- `--validate-feature-bounds` (every interval lies within its sequence)
- `--validate-translation` (CDS `/translation=` qualifiers agree with the translated sequence).

Switch `--validate` is the catch-all that runs all three; the individual `--validate-...` switches let you run them selectively. Each of these stops at the first violation, prints a two-line message pointing the user at `--validate-report`, and exits non-zero. For a complete enumeration, use the sibling `--validate-report <file>` action: it walks the whole register, writes one tab-separated row per violation (`check`, `path`, `feature_id`, `message`) to `<file>`, and exits non-zero only if at least one violation was found.

The tabular format

Alongside GFF3, GTF and GenBank, `AnnoTools` reads and writes a tabular format that is the register's *text twin*: it holds everything the binary `.Annotation` archive holds, but as plain tab-separated tables you can `diff`, `grep`, `sort` and `join`. It exists for two reasons. The first is that in GFF3 the hierarchy is *implicit*: the file carries `ID` and `Parent` edges between individual features, but never states the schema those edges are meant to satisfy — that a CDS may sit beneath an mRNA beneath a gene, and not elsewhere — so it cannot be read back without being told, from outside, what shape to expect. A tabular document states its hierarchy in its metadata table and so reads back knowing nothing about it, and it carries the resolved path on every feature row, so the nesting is something `awk` can filter on rather than something you reconstruct by walking a chain of edges. A schema you have to supply separately is a piece of the register the file does not hold.

The second is that GFF3 — the obvious candidate, being already a TSV — is awkward to manipulate with ordinary tools: its ninth column is a nested `key=v1,v2;key=v3` syntax that has to be re-parsed before you can `cut` anything out of it, its arity therefore depends on its content, so `cat`ting two files together is not meaningful and one feature with a novel attribute key reshapes nothing but reads differently, and a multi-valued or valueless attribute needs a sub-syntax rather than simply being a row. The tabular format puts each relation in the shape its consumers use, and gives every feature a content-derived identity that is stable when features are inserted, so either table can be sorted and still `join`ed.

Writing takes a *prefix* and produces three tables:

```
AnnoTools --from-genbank NC_000913.gb --to-tsv NC_000913
```

```
NC_000913.AnnotationFeatures.txt     #id #parent #seq #path #feature_id #source #score #strand #phase
#intervals
NC_000913.AnnotationAttributes.txt   #id #key #value
NC_000913.AnnotationMetadata.txt     #key #value
NC_000913.AnnotationReference.fasta  the sequence, as FASTA
```

Each table opens with a single line naming its columns, each column name prefixed with `#` as every tabular output in this family of tools spells a header. That one line is the whole of the framing: the three headers differ, so in the single-document form below the header *is* the table's identity and nothing else has to be agreed on.

None of the tables contains a nested syntax, and every one has a fixed number of columns, so `cat`ting two together is meaningful and adding a feature with a novel attribute key does not reshape any other row. Attributes live in their own relation rather than packed into a column, which is what lets a multi-valued attribute be several rows and a valueless one be a row with an empty third field. The metadata table carries the hierarchy the annotation was read under, plus any file-level pragmas, so a register written this way can be read back without being told its schema again.

The reference sequence travels too, as FASTA rather than as a fourth table — a sequence is not tabular data, and a 30 kbp cell would be the same category error as packing attributes into one column. This matters because a GenBank record is self-contained: without it, a GenBank → tabular pipeline would drop the sequence silently and every later `--extract-*` would fail for want of it. A register with no reference simply writes no FASTA. Per-sequence translation tables ride in the metadata table as `!table:<name>` rows, and only when they are not the standard code.

The sequence is written on a single line, the way `FASTools` writes sequence, so that every line of a tabular document is one whole record and `awk`, `cut` and `sort` can take it a line at a time. The `--extract-*` actions do the same. GFF3's `##FASTA` section is the exception, and stays wrapped at 60 because third-party GFF3 readers expect it; `--fasta-width <n>` overrides all three at once, with `0` meaning one line per sequence.

A feature's `id` is a content hash — 64-bit FNV-1a over its identity, chained through its parent — not a row number. It is therefore stable when features are inserted, and independent of row order: the `parent` column is what rebuilds the forest, so either table can be sorted and still `join`ed on the `id`. Chaining through the parent is what keeps the identical exons that alternative transcripts share distinguishable.

Reading takes the same prefix:

```
AnnoTools --from-tsv NC_000913 --to-genbank round-tripped.gb
```

For pipelines, a prefix under `/dev/*` — or a plain file that turns out to be one — selects a single-document form in which the tables simply follow one another, each recognised by its own `#` header, with the FASTA last. So `--to-tsv /dev/stdout` composes with the rest of a shell pipeline.

Submitting an annotation

`--to tbl` writes an NCBI submission feature table, the `.tbl` that `table2asn` consumes:

```
AnnoTools --from-genbank NC_000913.gb --to-tbl NC_000913.tbl
```

This format is *write-only*, and deliberately so rather than by omission: it has no slot for GFF3's source column, no parent link and no annotation metadata, and `table2asn` **infers** the gene/mRNA/CDS relations from coordinate overlap and shared identifiers instead of reading them. Inference is not encoding, so nothing can be recovered from one — which is why there is no `--from-tbl`, and why the type system rather than a runtime error is what tells you so.

It is also not a substitute for the tabular format above: a `.tbl` is a stanza format wearing a TSV costume — a block header, coordinate lines, and tab-indented qualifier continuation lines — so a feature's `locus_tag` sits several lines away from its coordinates and `awk` cannot operate on it per record. The two formats exist for different jobs.

One limitation worth stating plainly: a *filtered* tabular table does not read back. `Annotation.add` requires a feature's parent to be present, so `grep CDS` over the features table produces a file whose parents are missing and which is refused. Sorting is fine; filtering is not.

Selecting features and extracting their sequences

A *selection register* sits beside the annotation register and restricts `--selection-print` and the `--extract-*` actions to the features it matches — the same pattern, and the same switch names, that `KPopCountDB` uses to select spectra. A selection is expressed as a comma-separated list of `<field>~<regexp>` criteria, all of which must match:

```
AnnoTools --from-genbank NC_000913.gb -R 'type~CDS' --selection-print
```

A field is one of:

Field	Matches
`type`	the feature's own category, e.g. `CDS` or `mat_peptide`
`path`	its whole category chain, e.g. `source->CDS`
`seq`	the sequence it lies on
`strand`	`+`, `-` or `.`
`id`	its identifier (`label`, and the empty field name, are synonyms)
`source`	the provenance in GFF3 column 2

Any other name is read as an *attribute*, matching when any one of that attribute's values does — so `-R 'gene~dnaA'` selects on the `/gene` qualifier. Those names are therefore reserved: an attribute that happens to share one cannot be selected on.

The regexp is **unanchored**, which is the one thing worth remembering: `type~gene` also matches `pseudogene`. Anchor it with `^...$` when that matters.

So to pull every mature peptide out of a viral GenBank record:

```
AnnoTools --from-genbank NC_045512.gb -R 'type~^mat_peptide$' --extract-protein peptides.faa
```

and some further shapes:

```
AnnoTools ... -R 'type~^CDS$,gene~^thr'   # CDSs whose /gene starts with "thr" (criteria are ANDed)
AnnoTools ... -R 'seq~^chr1$'             # everything on one sequence
AnnoTools ... -R '~b0011'                 # the feature whose id is b0011
AnnoTools ... -R 'type~^CDS$' --selection-negate   # everything that is not a CDS
```

The sibling switch `-L` selects by a feature's **identifier** instead, matching exactly rather than by pattern:

```
AnnoTools --from-genbank NC_000913.gb -L b0011,b0012 --extract-dna genes.fasta
```

Where that identifier comes from depends on the source format — GFF3 takes it from `ID=`, GenBank from `/locus_tag` or else `/gene`, and GTF only gives one to the gene and transcript levels, from `gene_id` and `transcript_id`. The consequence is worth knowing: **many features have no identifier at all**, including every row of a GTF file and a GenBank `mat_peptide`, and `-L` can never match those. Select them with `-R` on `type` or `path`.

`--selection-print` is how you find out what to pass. Its first column is the identifier when the feature has one, and a positional stand-in of the form `<seq>:<type>:<location>` when it does not — that stand-in is a label, not an identifier, and feeding it back to `-L` matches nothing.

Under `-v`, every change to the selection reports what it now matches:

```
(AnnoTools): selection matches 1 of 4 features (regexps {type})
```

which is worth having because a selector is easy to get subtly wrong — an unanchored regexp, or a field name silently read as an attribute — and the first symptom is otherwise an output that is empty or far too large, by which point the action that produced it has already run. The same line is printed after an `--annotation` read, since a selection is a criterion rather than a fixed set and what it matches moves when the register does. The selection is sticky, starts out matching everything, and can be inverted with `--selection-negate` or reset with `--selection-clear`.

Given a reference, the selected features' sequences can then be written out as FASTA:

```
AnnoTools --from-genbank NC_000913.gb -R 'type~CDS' --extract-protein proteins.fasta
```

A feature's intervals are spliced in the order they are stored, so a `join(...)` location comes out as one record rather than one per exon; the result is reverse-complemented as a whole when the feature is on the minus strand. For `--extract-protein` the phase bases are dropped from the 5' end and the feature's `/transl_table` is honoured when it carries one. Because a GenBank file supplies its own sequence through its `ORIGIN` block, the example above needs nothing else; for GFF3 or GTF input, add `--from-fasta` first.

Command line options for `AnnoTools`

This is the full list of command line options available for the program `AnnoTools`. You can visualise the list by typing

```
AnnoTools -h
```

in your terminal. You will see a header containing information about the version:

```
This is AnnoTools version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2026 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
AnnoTools [ACTIONS]
```

Actions. They are executed delayed and in order of specification.

Operations on the annotation register:

Option	Argument(s)	Effect	Note(s)
`-0` `--empty`		load an empty annotation into the register	
`-i` `--input`	*<binary_file_prefix>*	load into the register the annotation present in the specified binary file (extension '.Annotation' is appended unless the path is under '/dev/*')	
`-o` `--output`	*<binary_file_prefix>*	write the current register to the specified binary file (extension '.Annotation' is appended unless under '/dev/*')	

Hierarchy. Override the active hierarchy for a given format. The override is sticky: every subsequent input operation in that format uses it until another '--hierarchy' or '--dialect' replaces it. Reverting to the format's default is just '--dialect <fmt> standard'.

Option	Argument(s)	Effect	Note(s)
`--hierarchy`	*<gff3|gtf|genbank>* *<S-expression>*	set the hierarchy to use for subsequent input operations in the named format	
`--dialect`	*<gff3|gtf|genbank>* *<name>*	switch subsequent input operations in the named format to one of its built-in dialects. Currently only GFF3 ships more than one dialect ('standard' and 'gencode').	

Annotation input. Long form: action mode + format + path. Short forms: '--from-gff3', '--from-gtf', '--from-genbank' default to 'replace'.

Option	Argument(s)	Effect	Note(s)
`-a` `--annotation`	*<replace|add>* *<gff3|gtf|genbank|tsv>* *<file_or_prefix>*	merge or replace the register from the named format. Every format but 'tsv' takes a FILE; 'tsv' takes a PREFIX, since it is a collection of files. When the format is GenBank and the input carries an ORIGIN section, or GFF3 and it carries a '##FASTA' section, the reference sequence is replaced as well.	
`--from-gff3`	*<file>*	shorthand for '--annotation replace gff3 <file>'	
`--from-gtf`	*<file>*	shorthand for '--annotation replace gtf <file>'	
`--from-tsv` `--from-tabular`	*<prefix>*	shorthand for '--annotation replace tsv <prefix>'. Reads the '.AnnotationFeatures.txt', '.AnnotationAttributes.txt' and '.AnnotationMetadata.txt' tables written from that prefix, together with '.AnnotationReference.fasta' when one is beside them. A path under '/dev/*', or an ordinary file that turns out to be a whole tabular document, is read as one document instead	
`--from-genbank`	*<file>*	shorthand for '--annotation replace genbank <file>'	

Reference (multi-FASTA) input. Long form takes the same mode keyword as --annotation. Short form '--from-fasta' defaults to 'replace'.

Option	Argument(s)	Effect	Note(s)
`-r` `--reference`	*<replace|add> <file>*	merge or replace the register's reference from <file>	
`--from-fasta`	*<file>*	shorthand for '--reference replace <file>'	

Validation. Each check stops at the first violation, exits non-zero, and points the user at '--validate-report <file>' for the full list. All require a reference to be set.

Option	Argument(s)	Effect	Note(s)
`--validate-sequences-present`		every sequence referenced by an annotation feature must also exist in the reference	
`--validate-feature-bounds`		every feature interval must lie within the corresponding sequence's length	
`--validate-translation`		translated CDS features must agree with their /translation= qualifier (currently a structural sub-check; codon-by-codon comparison is a follow-up)	
`--validate`		run every validation in turn	
`--validate-report`	*<file>*	run every validation against the current register but do not stop at the first violation: walk the whole register, write a tab-separated report with one row per violation (columns: check, path, feature_id, message) to <file>, and exit non-zero if any violation was found.	

Actions involving the selection register. The selection restricts '--selection-print' and the '--extract-*' actions to the features it matches. The '--to-*' writers and '-o' always write the whole register, because a feature whose parent is not selected would be emitted without it. The selection is sticky, and starts out matching everything. To pull every mature peptide out of a GenBank record: AnnoTools --from-genbank in.gb \ -R 'type~^mat_peptide$' \ --extract-protein peptides.faa Add '-v' to see how many features each selection matched.

Option	Argument(s)	Effect	Note(s)
`-L` `--labels` `--selection-from-labels`	*<feature_id> ['<feature_id>]*	put into the selection register the features carrying the given identifiers. The match is EXACT, not a regexp: for patterns use '-R' with the 'id' field. A feature's identifier comes from its source format: GFF3 the 'ID=' attribute GenBank '/locus_tag', or '/gene' when there is none GTF only the gene and transcript levels, from 'gene_id' and 'transcript_id' Many features have NO identifier -- a GenBank mat_peptide, and every row of a GTF file, since there only the synthesised gene and transcript parents get one. '-L' can never match those; select them with '-R' on 'type' or 'path' instead. '--selection-print' lists them, which is how to find out what to pass. Its first column is the identifier when the feature has one and a positional stand-in of the form '<seq>:<type>:<location>' when it does not -- the latter is a label, not an identifier, and '-L' will not match it. Examples: -L b0011 one feature, by locus tag -L b0011,b0012,b0013 three of them -L ENSG00000141510 a GFF3 feature by its ID=	
`-R` `--regexps`	*<field>'~'<regexp> ['<field>'~'<regexp>]*	put into the selection register the features whose named fields match the given regexps. Criteria separated by ',' must ALL	

Option	Argument(s)	Effect	Note(s)
`--selection-from-regexps`		match. <field> is one of: type the feature's own category, e.g. CDS, mat_peptide path its whole category chain, e.g. source->CDS seq the sequence it lies on strand '+', '-' or '.' id its identifier ('label', and the empty field name, are synonyms) source the provenance in GFF3 column 2 Any other name is read as an ATTRIBUTE, matching when any one of that attribute's values does -- so 'gene~dnaA' selects on the /gene qualifier. Those seven names are therefore reserved: an attribute sharing one of them cannot be selected on. <regexp> is UNANCHORED, so 'type~gene' also matches 'pseudogene'. Anchor it with '^...$' when that matters. Examples: -R 'type~^mat_peptide$' every mature peptide -R 'type~^CDS$,gene~^thr' CDSs whose /gene starts 'thr' -R 'seq~^chr1$' everything on chr1 -R '~b0011' the feature whose id is b0011	
`--selection-negate`		negate the current selection	
`--selection-print`		print the features currently selected, one per line, to standard output	
`--selection-clear`		reset the selection register so that it matches everything	

Sequence extraction. Emit the sequence denoted by each selected feature as FASTA. A feature's intervals are spliced in the order they are stored and the result is reverse-complemented when the feature is on the minus strand; a protein is that sequence with the phase bases dropped from its 5' end, translated with the feature's '/transl_table' when it carries one. Requires a reference to have been loaded. Each defline names the feature and then carries, as a bracketed '[key=value]' apiece, where it came from -- 'path', 'seq', 'location' -- and every qualifier it holds. The brackets are in the manner of NCBI's own extracts, and they are what keeps the line splittable: both a qualifier such as 'product=hypothetical protein' and a sequence name may carry spaces. 'ID' and 'Parent' are left out, being the name and the path over again.

Option	Argument(s)	Effect	Note(s)
`--extract`	*<dna|protein>* *<file>*	write the sequence of every selected feature to <file>	
`--extract-dna`	*<file>*	shorthand for '--extract dna <file>'	
`--extract-protein`	*<file>*	shorthand for '--extract protein <file>'	
`--summary`		print a one-line summary of the current register to stderr	

Annotation output.

Option	Argument(s)	Effect	Note(s)
`--to`	*<gff3|gtf|genbank|tsv|tbl>* *<file_or_prefix>*	write the register in the named format. Every format but 'tsv' takes a FILE; 'tsv' takes a PREFIX, since it writes a collection of files. 'tbl' is NCBI's submission feature table, which is write-only: it encodes no hierarchy and no metadata, so nothing can be read back from it	
`--to-gff3`	*<file>*	shorthand for '--to gff3 <file>'	
`--to-gtf`	*<file>*	shorthand for '--to gtf <file>'	
`--to-tsv` `--to-tabular`	*<prefix>*	shorthand for '--to tsv <prefix>'. Writes a COLLECTION of files: the '.AnnotationFeatures.txt', '.AnnotationAttributes.txt' and '.AnnotationMetadata.txt' tables, plus '.AnnotationReference.fasta' when the register carries a sequence.	

Option	Argument(s)	Effect	Note(s)
		A prefix under '/dev/*' writes all of it to that one path instead, as a single document	
`--to-tbl` `--to-feature-table`	*<file>*	shorthand for '--to tbl <file>'	
`--to-genbank`	*<file>*	shorthand for '--to genbank <file>'	

Miscellaneous options. They are set immediately.

Option	Argument(s)	Effect	Note(s)
`--fasta-width`	*<non_negative_integer>*	wrap sequence lines at this width wherever FASTA is emitted ('--to-gff3', '--to-tsv' and the '--extract-*' actions), with '0' meaning one line per sequence. Without this option each format keeps its own convention: GFF3 wraps its '##FASTA' section at 60, while the tabular format and the extraction actions emit one line per sequence, so that every line of their output is a whole record	default= *each format's own convention*
`-v` `--verbose`		set verbose execution	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `TREx`

`TREx` finds exact tandem repeats in all the sequences present in a FASTA file read on standard input, and writes a tab-separated record per identified locus to standard output (sequence name, start and end coordinates, unit length, repeat count, and the repeat unit itself). The output is convenient for downstream filtering by length, intersection against an annotation, or visualisation as a per-sequence track.

The input alphabet is configured by `--linter` (default `dna`); the algorithm-tuning options `--minimum_locus_length` and `--maximum_repeat_length` bound the search. A typical invocation requiring loci of at least 50 bp and unit length up to 10 nt looks like

```
TREx -m 50 -M 10 < genome.fasta > repeats.tsv
```

Command line options for `TREx`

```
This is TREx version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2023-2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

Usage:

```
TREx [OPTIONS]
```

Input/Output

Option	Argument(s)	Effect	Note(s)
`-l` `--linter`	*'none'|'DNA'|'dna'|'protein'*	sets linter for sequence. All non-base (for DNA) or non-AA (for protein) characters are converted to unknowns	default= *dna*
`--linter-keep-lowercase`	*<bool>*	sets whether the linter should keep lowercase DNA/protein characters appearing in sequences rather than capitalise them	default= *false*
`--linter-keep-dashes`	*<bool>*	sets whether the linter should keep dashes appearing in sequences rather than convert them to unknowns	default= *false*

Algorithm

Option	Argument(s)	Effect	Note(s)
`-M` `--maximum_repeat_length`	*<non_negative_integer>*	maximum unit length for a tandem repeat to be considered	default= *4611686018427387903*
`-m` `--minimum_locus_length`	*<non_negative_integer>*	minimum locus length for a tandem repeat to be considered	default= *0*

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-v` `--verbose`		set verbose execution (global option)	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `Cophenetic`

`Cophenetic` reads a Newick tree on standard input and writes the cophenetic-distance matrix between every pair of labelled nodes (leaves and internal alike) to standard output, as a tab-separated table with row and column headers. By default the distances are shortest-path along branch lengths — the standard cophenetic interpretation. The `-M` / `--max-distances` switch produces the longest-path matrix instead, which is the right convention when branch lengths encode partial quantities such as allele frequencies (so two siblings of a common ancestor with branch lengths 0.3 and 0.5 sit at "distance" 0.5 rather than 0.8). The matrix computation is parallelised across the cores chosen by `-t` / `--threads`.

```
Cophenetic -t 4 < tree.nwk > distances.tsv
```

Command line options for `Cophenetic`

```
This is Cophenetic version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

Usage:

```
Cophenetic [OPTIONS]
```

Algorithm

Option	Argument(s)	Effect	Note(s)

`-M`
`--max-distances`
`--maximum-distances`
`--longest-path-distances`
		compute longest- rather than shortest-path distances	default= *compute shortest-path distances*

Miscellaneous

Option	Argument(s)	Effect	Note(s)

`-t`
`-T`
`--threads`
	<*computing_threads*>	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= *4*

`-v` `--verbose`		set verbose execution (global option)	default= *false*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `NJ`

`NJ` reads a matrix of pairwise distances and writes the neighbour-joining tree those distances imply. It is the counterpart of `Cophenetic`, and reads back the same tab-separated matrix format the rest of this suite writes — a header line of column names preceded by an empty field, then one row per name. Both axes must carry the same names in the same order.

```
NJ -i distances.tsv -o tree.nwk
```

The algorithm is Saitou and Nei's, in Studier and Keppler's formulation, which is what makes each of the n joining steps cost $O(n^2)$ rather than $O(n^3)$: a matrix of 764 taxa goes through in about a second, reading included. Given an additive matrix — one that some tree's branch lengths generated — it recovers that tree exactly, branch by branch.

The tree it writes is **unrooted**, as neighbour joining's is: the last three subtrees are resolved in closed form into a trifurcation, so the node Newick has to put first adds no bipartition of its own. `-m` / `--midpoint` re-roots the tree at the midpoint of its longest tip-to-tip path — the point equidistant from the two most distant tips — which is what one roots at when no outgroup is available:

```
NJ -m -i distances.tsv -o rooted.nwk
```

Rooting moves no distance between tips and neither creates nor loses branch length: the edge holding the midpoint is split in two, and a node the re-rooting leaves with one parent, one child and no name of its own — the old root of a tree that was already rooted — is spliced out with its two edges merged. Rooting an already-rooted tree is therefore rooting it once.

Two things a real distance matrix tends to do that a textbook one does not:

- it is often not quite symmetric. `-a` / `--asymmetry` chooses between replacing both cells with their mean (`average`, the default) and refusing the matrix (`error`). Under `-v` the largest disagreement found is reported together with the pair it was found on, so that "not quite" can be checked rather than assumed.
- it is often not additive, and neighbour joining then yields branches of negative length. `-n` / `--negative-branches` chooses between keeping them (`ok`, the default — the undefined-length sentinel is not a negative number, so they are unambiguous, and they measure how far from additive the matrix is), flattening them to zero (`zero`), and refusing the matrix (`error`).

Both are things to check rather than assume, which is what `-S` / `--statistics` is for: it writes a two-line table describing the matrix itself — its shape, whether it can be joined at all, how far it disagrees with itself across the diagonal (on average, at worst, and on which pair), how far its diagonal is from zero, how many of its cells are negative, its range, and, for the tree, how many branches came out negative and what the branches add up to.

```
NJ -S stats.tsv -i distances.tsv -o tree.nwk
```

It is written **even for a matrix that is then refused**, which is when a description is worth the most: a table reading `#rows 764`, `#columns 3` explains the refusal that follows it, where the refusal alone only reports it. That also makes it the natural way to survey a family of matrices and pick one worth joining — run `NJ -S` over each and read down the negative-branch column.

The output is plain Newick, which is what most other programs expect. `-r` / `--rich-format` emits the dialect this suite understands instead, which tags the tree as rooted (`[&R]`) or unrooted (`[&U]`).

Command line options for `NJ`

```
This is NJ version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2026 Paolo Ribeca <paolo.ribeca@gmail.com>
```

Usage:

```
NJ [OPTIONS]
```

Input/Output. The distance matrix is the tab-separated form the rest of this suite reads and writes: a header line of column names preceded by an empty field, and one row per name. Both axes must carry the same names in the same order

Option	Argument(s)	Effect	Note(s)
`-i` `--input`	*<distance_matrix>*	name of the file containing the matrix of pairwise distances	default= */dev/stdin*
`-o` `--output`	*<newick_file>*	name of the file the resulting tree should be written to	default= */dev/stdout*
`-S` `--statistics`	*<statistics_file>*	also write a two-line table of what the distance matrix is: its shape, whether it can be joined at all, how far it disagrees with itself across the diagonal and where, how many cells are negative (a distance is not, so those are however the matrix writes 'not measured'), and, for the tree, how many branches came out negative and what they add up to. Written even for a matrix that is then refused, that being when a description is worth the most	
`-r` `--rich-format`		emit the rich Newick dialect this suite understands, which tags the tree as rooted or unrooted and carries dictionaries and hybrid nodes; plain Newick is what most other programs expect	default= *false*

Algorithm

Option	Argument(s)	Effect	Note(s)
`-m` `--midpoint` `--midpoint-root`		re-root the tree at the midpoint of its longest tip-to-tip path, rather than leaving it unrooted as neighbour joining produces it	default= *false*
`-a` `--asymmetry`	*'average'|'error'*	what to do when the matrix disagrees with itself across the diagonal: replace both cells with their mean, or refuse the matrix	default= *average*

Option	Argument(s)	Effect	Note(s)
`-n` `--negative-branches`	*'ok'|'zero'|'error'*	what to do about the branches of negative length a matrix that is not additive yields: keep them, flatten them to zero, or refuse the matrix	default= *ok*

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-t` `-T` `--threads`	*<computing_threads>*	number of concurrent computing threads to be spawned (used when reading the matrix; the joining itself is sequential) (default automatically detected from your configuration)	default= *4*
`-v` `--verbose`		set verbose execution (global option)	default= *false*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

Using `Yggdrasill`

`Yggdrasill` builds an unrooted phylogenetic tree from a register of weighted splits. The natural input is a `.PhyloSplits` file produced by `KPopTwistDB`'s `--splits` action, but any tool that emits the same format can drive it. The action-language semantics mirrors `KPopCountDB` and `KPopTwistDB`: each command-line flag enqueues one register operation, the operations are executed in command-line order, and combinations of `-i` / `-I` / `-a` / `-A` / `-t` / `-o` / `-O` let you load, merge, build trees from, and dump splits in any order.

The central operation is `-t` / `--tree <prefix>`: it reads the live splits register, greedy-filters the splits in order of decreasing weight for pairwise compatibility, assembles the accepted subset into a Newick tree via the Buneman construction, and writes the tree to `<prefix>.nwk` and the compatible-splits subset to `<prefix>.PhyloSplits`. The incompatible residual is left in the register, so a subsequent `-t` pass can build a "tree of residuals", or the residual can be written out via `-o` / `-O` for inspection.

After every `-t` pass, a one-line summary is emitted to stderr reporting how many compatible splits were accepted, how many were dropped as incompatible, the total weight on each side, and the ratio of dropped to total weight. That ratio is a direct measure of how tree-like the input split system was: values near zero indicate the data is essentially tree-like, while values approaching one signal genuine network-like structure that the Buneman construction necessarily projects away.

A typical end-to-end pipeline taking a `KPopTwistDB` splits output and producing both the tree and the residual is

```
KPopTwistDB -i T train -i t train --splits-algorithm gaps -S splits
Yggdrasill  -i splits -t tree -O dropped
```

Command line options for `Yggdrasill`

```
This is Yggdrasill version 1.3.3-851 [25-Aug-2026]
 compiled against: BiOCamLib version 1.3.3-851 [25-Aug-2026]
 (c) 2024-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
```

Usage:

```
Yggdrasill [OPTIONS]
```

Actions. They are executed delayed and in order of specification.

Option	Argument(s)	Effect	Note(s)
`-c` `--clear`		clear the splits register (keep names, discard existing splits)	
`-i` `--input`	*<binary_file_prefix>*	load into the splits register the specified binary database (which must have extension '.PhyloSplits' unless file is '/dev/*')	
`-I` `--Input`	*<splits_file_prefix>*	load into the splits register the specified plain text database (which must have extension '.PhyloSplits.txt' unless file is '/dev/*')	
`-a` `--add`	*<binary_file_prefix>*	add to the contents of the splits register the specified binary database (which must have extension '.PhyloSplits' unless file is '/dev/*')	
`-A` `--Add`	*<splits_file_prefix>*	add to the contents of the splits register the specified plain text database (which must have extension '.PhyloSplits.txt' unless file is '/dev/*')	
`--input-tree` `--Of-tree`	*<newick_file>*	load into the splits register the bipartitions of the specified Newick tree (replaces the current register; for multi-tree files, every tree's bipartitions are added, each with weight 1.0 per occurrence)	
`--Add-tree`	*<newick_file>*	add to the splits register the bipartitions of the specified Newick tree, each with weight 1.0 (so a bipartition supported by k trees accumulates to weight k). Compose with multiple --Add-tree calls for ensemble consensus, then feed to -t	
`--Add-tree-weighted`	*<newick_file>* *<weight>*	as --Add-tree, but each bipartition gets the explicit weight given (useful when different ensemble members should count differently)	
`--drop-weak-splits`	*<float>*	drop every split in the register whose accumulated weight is strictly less than the cutoff. Apply between --Add-tree calls and -t to implement majority-rule consensus at threshold p: set cutoff = p * n_input_trees	
`--negative-branches-policy`	*'error'|'ok'|'zero'*	policy for handling negative branch lengths when reading Newick input (via --input-tree, --Add-tree, --Add-tree-weighted). 'error' rejects (raises a parse error); 'ok' accepts as-is; 'zero' silently clamps to 0. Set before the --Add-tree call(s) it should apply to	default= *ok*
`-t` `--tree`	*<tree_file_prefix>*	generate a phylogenetic tree from the contents of the splits register. The results will be a Newick file (which will be given extension '.nwk' unless file is '/dev/*') and the database of compatible splits used to build the tree (which will be given extension '.PhyloSplits.txt' unless file is '/dev/*'). The residual incompatible splits will be moved back to the splits register	
`-o` `--output`	*<binary_file_prefix>*	dump the contents of the splits register to the specified binary file (which will be given extension '.PhyloSplits' unless file is '/dev/*')	
`--precision`	*<positive_integer>*	set the number of precision digits to be used when outputting numbers	default= *10*
`-O` `--Output`	*<splits_file_prefix>*	dump the contents of the splits register to the specified plain text file (which will be given extension '.PhyloSplits.txt' unless file is '/dev/*')	

Miscellaneous options. They are set immediately.

Option	Argument(s)	Effect	Note(s)
`-v` `--verbose`		set verbose execution (global option)	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	